Meeting Notes

Project Title: Compress-Transfer-Decompress for LLM Serving: cuSZp-Enabled CPU-GPU Data Pipeline in vLLM

Student: KONG Zirui

Supervisor: Dr. ZHANG Zhaorui

Meeting 1: Interim Review & Transition to Real Model Integration

Date: Feb 18, 2026

Location: Online

Meeting Type: Mid-term Progress Review

1. Review of Interim Status:

- Demonstrated the Python-based simulated environment (`test_vllm_integration.py`).
- The custom `MockCacheEngine` successfully validated the Monkey-Patching mechanics. The python hooks are correctly intercepting the simulated `_swap_out` and `_swap_in` memory routines.
- Pseudo-random numbers were compressed and restored within the allowed mathematical error bounds, proving the theoretical feasibility of the "Compress-Transfer-Decompress" pipeline.

2. Supervisor Feedback & Discussion:

- **Critique:** While the simulation confirms the logical flow, testing memory bandwidth with randomized floating-point noise does not accurately reflect how a language model behaves in production. The inherent sparsity and distribution of true KV cache tensors are fundamentally different from random noise.
- **Direction:** The project must immediately pivot from synthetic simulations to integrating real-world models.

3. Action Items:

- Integrate HuggingFace `transformers` library to extract true Layer-0 key embeddings (`past_key_values[0][0]`).
- Prepare evaluation targets including Qwen2.5, GPT-2, and OPT architectures.
- Begin drafting the C++ wrapper for the cuSZp compression algorithm using PyBind11 to replace the simulated python compression logic.

Meeting 2: C++ Backend Integration & Memory Safety Debugging

Date: Mar 4, 2026

Location: Online

1. Progress Update:

- The initial C++ backend (`cuszp_wrapper.cpp`) has been drafted.

- Successfully implemented the dynamic error-bound calculation using CUDA reduction primitives ($\epsilon_{abs} = \epsilon_{rel} \times \max(R, 10^{-6})$) to accommodate varying tensor magnitudes across different LLM layers.
- The `compressed_blocks_store` dictionary is actively logging reconstructed metadata.

2. Technical Challenges Discussed:

- **Segmentation Faults:** Encountered severe data corruption and memory access violations when passing Python tensors to the C++ backend.
  - *Root Cause:* vLLM's PagedAttention dynamically slices and reshapes matrices, meaning the underlying PyTorch tensors are often no longer stored in contiguous physical memory blocks.
  - *Solution:* Enforce strict `.contiguous()` calls on all target tensors before extracting raw pointers for C/CUDA extensions.
- **Race Conditions:** Decompression kernels occasionally fired before the byte stream fully arrived in VRAM.
  - *Solution:* Extract vLLM's active PyTorch CUDA stream via `c10::cuda::getCurrentCUDAStream()` and pass it through to the cuSZp library to guarantee proper asynchronous process synchronization.

3. Action Items:

- Containerize the entire execution environment using Docker and the NVIDIA Container Toolkit to resolve current WSL2-to-Linux cross-platform compilation inconsistencies (specifically CRLF vs. LF shell execution bugs in `run.sh`).
- Deploy the pipeline onto the NVIDIA RTX 5080 workstation for preliminary physical hardware testing.

Meeting 3: Benchmarking Pipeline & Sensitivity Analysis

Date: Mar 20, 2026

Location: Online

1. Progress Update:

- The Docker container is fully operational. The build script automatically clones the official cuSZp GitHub repository and compiles it globally.
- An automated benchmarking script (`benchmark_pipeline.py`) has been developed. It uses a repeated dense informational prompt (*"The Hong Kong Polytechnic University..."*) to generate realistic long-context hidden states.
- The payload sizes are successfully standardized to $4,194,304$ elements (16 MB FP32) to ensure equitable comparisons across architectures.

2. Discussion on Error Bounds & Trade-offs:

- Conducted a sensitivity sweep on GPT-2 and Qwen2.5, shifting the relative error bound (ϵ_{rel}) incrementally from 10^{-2} to 10^{-5}.
- *Observation:* Loosening the bound to 10^{-2} yielded a massive 8.25$\times$ compression ratio for Qwen2.5. However, this caused the absolute maximum error to spike above 6.0.

- *System Architecture Analysis:* Because LLMs utilize the exponential Softmax function for attention weights, an absolute error of 6.0 is exponentially amplified, destroying the autoregressive generation quality and causing severe hallucinations.
- *Conclusion:* Tightening the bound below 10^{-4} drops the compression ratio to $\sim 2.10\times$ with diminishing accuracy returns. The 10^{-4} threshold perfectly represents the Pareto optimal point.

3. Action Items:

- Set 10^{-4} as the default baseline configuration for the final benchmark runs.
- Execute 50 active iterations (with explicit `torch.cuda.synchronize()` barriers) to average out robust runtime measurements for all 8 target models.

Meeting 4: Final Results Evaluation & Thesis Finalization

Date: Apr 3, 2026

Location: Online

Meeting Type: Final Sign-off

1. Final Benchmark Review:

- The implementation yielded exceptional performance metrics. Across the board, the pipeline shrank the physical data payload by factors of 2.42× to 3.06×.
- The RTX 5080 hardware masked the computational overhead efficiently. The execution time of the cuSZp compression kernel remained flat at ~ 0.8 ms, confirming the bottleneck was shifted from algorithmic branch divergence to raw memory interface throughput.
- **Peak Achievement:** The Qwen2.5-1.5B model achieved a 2.08× Swap-In speedup, effectively increasing the perceived PCIe interconnect bandwidth from 16.32 GB/s to 33.86 GB/s.

2. Limitations Discussed (For Future Work):

- Acknowledged the transient VRAM overhead required for cuSZp workspace buffers.
- Discussed diminishing returns on highly latency-sensitive, single-batch inferences due to fixed CUDA kernel launch overheads.
- Identified potential future research directions: Dynamic/Adaptive error bound tuning based on layer-specific sensitivity, and overlapping PCIe chunk pipelining.

3. Action Items:

- Export execution traces into JSON artifacts and `benchmark_summary.md`.
- Generate vector graphics using `pgfplots` (grouped bar charts for bandwidth and dual Y-axis line charts for sensitivity analysis) to insert into the LaTeX manuscript.
- Finalize the formatting of the LaTeX thesis manuscript and prepare for the final defense presentation ahead of the April 10 submission deadline.